Pet Guardian

Mastering Relational and Non-Relational Database - Sprint 4

Enzo Okuizumi RM 561432

Gustavo Okada RM 563428

Lucas Barros RM 566422

Luna Guimarães RM 562290

Milton Marcelino RM 564836

Links Importantes **3**
Modelo Lógico **4**
Modelo Relacional **5**
Tabelas Criadas no Oracle SQL Developer **6**
DML - Cargas de Dados **15**
Objetos PL/SQL **16**
Empacotamento de Processos: Package PKG_PETGUARDIAN 16
Exceções Personalizadas Tratadas no Pacote 17
Bateria de Testes e Evidências de Execução (PKG_PETGUARDIAN) **30**
Resultados da Bateria de Testes Integrada (Testes 1 a 5) 30
Banco de Dados Não Relacional — MongoDB (conteudos_aula) **32**
Persistência Poliglota e Modelagem Orientada a Documentos 32
1. Dataset Estruturado em JSON (conteudos_dataset.json) 32
2. Script de Criação da Coleção, Validação de Schema, Índices e Carga (setup_mongodb_conteudos.js) 35
3. Evidência de Execução e Importação no MongoDB (MONGOSH) 42

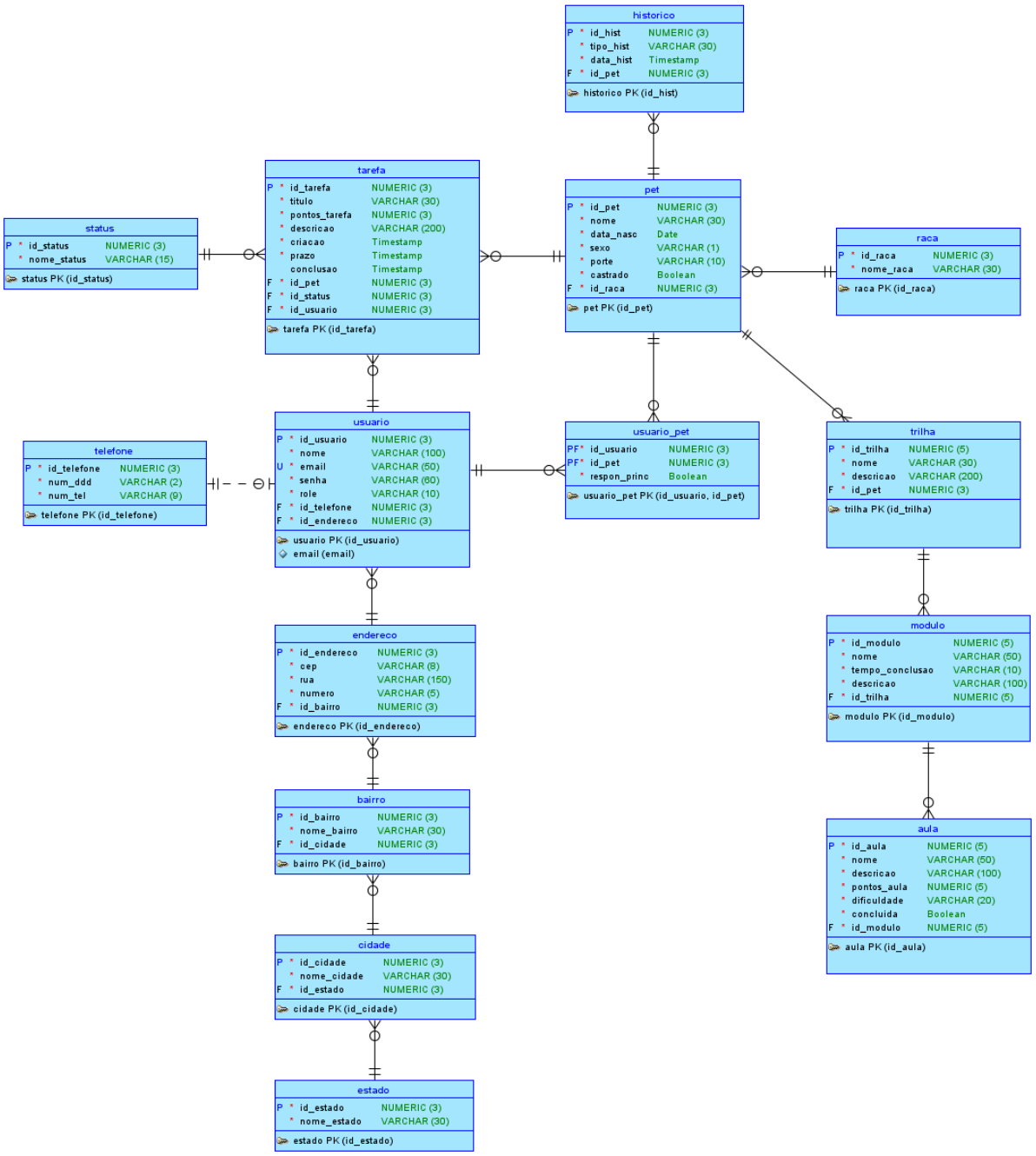
Links Importantes

Repositório Github: Challenge-Pet-Guardian-4/Database-Advanced

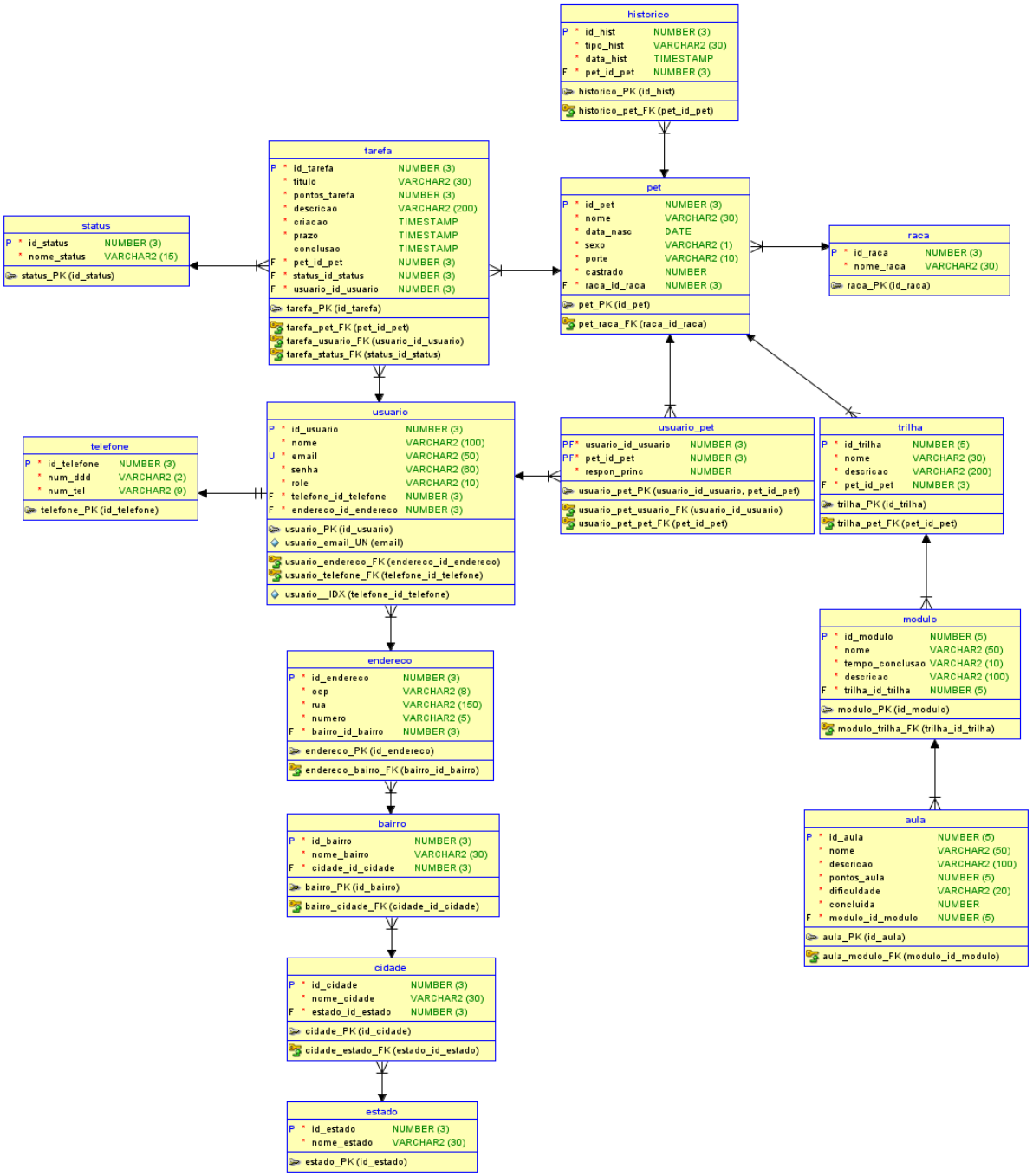
Link Vídeo Youtube:

Responsáveis pelo desenvolvimento: Lucas Barros / Enzo Okuizumi

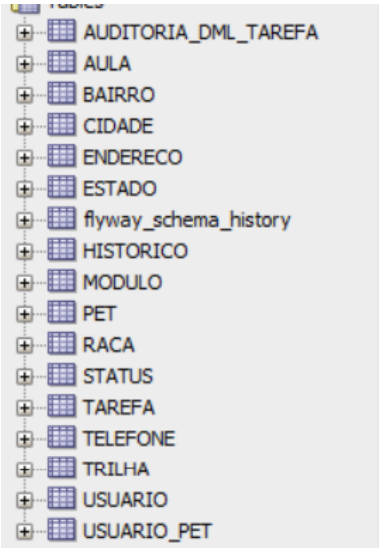
Modelo Lógico



Modelo Relacional



Tabelas Criadas no Oracle SQL Developer




```
Table TELEFONE created.


Table ESTADO created.


Table CIDADE created.


Table BAIRRO created.


Table ENDERECO created.


Table USUARIO created.


INDEX USUARIO__IDX created.


Table RACA created.

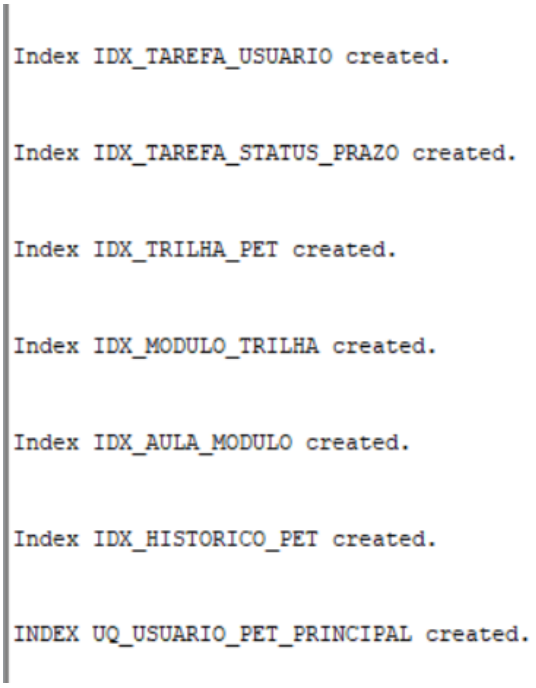

Table PET created.


Table USUARIO_PET created.


Table STATUS created.


Table TAREFA created.
```


Script Output
Task completed in 1,161 seconds
Table MODULO created.
Table AULA created.
Table HISTORICO created.
Table AUDITORIA_DML_TAREFA created.
Index IDX_CIDADE_ESTADO created.
Index IDX_BAIRRO_CIDADE created.
Index IDX_ENDERECO_BAIRRO created.
Index IDX_USUARIO_ENDERECO created.
Index IDX_PET_RACA created.
Index IDX_USUARIO_PET_PET created.
Index IDX_TAREFA_PET created.
Index IDX_TAREFA_STATUS created.


Index IDX_TAREFA_USUARIO created.
Index IDX_TAREFA_STATUS_PRAZO created.
Index IDX_TRILHA_PET created.
Index IDX_MODULO_TRILHA created.
Index IDX_AULA_MODULO created.
Index IDX_HISTORICO_PET created.
INDEX UQ_USUARIO_PET_PRINCIPAL created.

SQL

```sql
-- 1. TELEFONE
CREATE TABLE telefone (
    id_telefone NUMBER(3) GENERATED BY DEFAULT ON NULL AS
IDENTITY NOT NULL,
    num_ddd     VARCHAR2(2) NOT NULL,
    num_tel     VARCHAR2(9) NOT NULL,
    CONSTRAINT telefone_pk PRIMARY KEY (id_telefone)
);

-- 2. ESTADO
CREATE TABLE estado (
    id_estado   NUMBER(3) GENERATED BY DEFAULT ON NULL AS
IDENTITY NOT NULL,
    nome_estado VARCHAR2(30) NOT NULL,
    CONSTRAINT estado_pk PRIMARY KEY (id_estado)
);

-- 3. CIDADE
CREATE TABLE cidade (
    id_cidade        NUMBER(3) GENERATED BY DEFAULT ON NULL AS
IDENTITY NOT NULL,
    nome_cidade      VARCHAR2(30) NOT NULL,
    estado_id_estado NUMBER(3) NOT NULL,
    CONSTRAINT cidade_pk PRIMARY KEY (id_cidade),
    CONSTRAINT cidade_estado_fk FOREIGN KEY (estado_id_estado)
REFERENCES estado(id_estado)
);

-- 4. BAIRRO
CREATE TABLE bairro (
    id_bairro        NUMBER(3) GENERATED BY DEFAULT ON NULL AS
IDENTITY NOT NULL,
    nome_bairro      VARCHAR2(30) NOT NULL,
    cidade_id_cidade NUMBER(3) NOT NULL,
    CONSTRAINT bairro_pk PRIMARY KEY (id_bairro),
    CONSTRAINT bairro_cidade_fk FOREIGN KEY (cidade_id_cidade)
REFERENCES cidade(id_cidade)
);
```

```sql
-- 5. ENDERECO
CREATE TABLE endereco (
    id_endereco      NUMBER(3) GENERATED BY DEFAULT ON NULL AS
IDENTITY NOT NULL,
    cep              VARCHAR2(8) NOT NULL,
    rua              VARCHAR2(150) NOT NULL,
    numero           VARCHAR2(5) NOT NULL,
    bairro_id_bairro NUMBER(3) NOT NULL,
    CONSTRAINT endereco_pk PRIMARY KEY (id_endereco),
    CONSTRAINT endereco_bairro_fk FOREIGN KEY (bairro_id_bairro)
REFERENCES bairro(id_bairro)
);

-- 6. USUARIO (1:N com ENDERECO via FK endereco_id_endereco)
CREATE TABLE usuario (
    id_usuario           NUMBER(3) GENERATED BY DEFAULT ON NULL
AS IDENTITY NOT NULL,
    nome                 VARCHAR2(100) NOT NULL,
    email                VARCHAR2(50) NOT NULL,
    senha                VARCHAR2(60) NOT NULL,
    role                 VARCHAR2(10) NOT NULL,
    telefone_id_telefone NUMBER(3) NOT NULL,
    endereco_id_endereco NUMBER(3) NOT NULL,
    CONSTRAINT usuario_pk PRIMARY KEY (id_usuario),
    CONSTRAINT usuario_email_un UNIQUE (email),
    CONSTRAINT ck_usuario_role CHECK (role IN ('ADMIN', 'COMUM',
'PREMIUM')),
    CONSTRAINT usuario_telefone_fk FOREIGN KEY
(telefone_id_telefone) REFERENCES telefone(id_telefone),
    CONSTRAINT usuario_endereco_fk FOREIGN KEY
(endereco_id_endereco) REFERENCES endereco(id_endereco)
);
CREATE UNIQUE INDEX usuario__idx ON usuario(telefone_id_telefone
ASC);

-- 7. RACA
CREATE TABLE raca (
    id_raca   NUMBER(3) GENERATED BY DEFAULT ON NULL AS IDENTITY
NOT NULL,
```

```
    nome_raca VARCHAR2(30) NOT NULL,
    CONSTRAINT raca_pk PRIMARY KEY (id_raca)
);

-- 8. PET
CREATE TABLE pet (
    id_pet       NUMBER(3) GENERATED BY DEFAULT ON NULL AS
IDENTITY NOT NULL,
    nome         VARCHAR2(30) NOT NULL,
    data_nasc    DATE NOT NULL,
    sexo         VARCHAR2(1) NOT NULL,
    porte        VARCHAR2(10) NOT NULL,
    castrado     NUMBER NOT NULL,
    raca_id_raca NUMBER(3) NOT NULL,
    CONSTRAINT pet_pk PRIMARY KEY (id_pet),
    CONSTRAINT ck_pet_sexo CHECK (sexo IN ('F', 'M')),
    CONSTRAINT ck_pet_porte CHECK (porte IN ('GRANDE', 'MEDIO',
'PEQUENO')),
    CONSTRAINT ck_pet_castrado CHECK (castrado IN (0, 1)),
    CONSTRAINT pet_raca_fk FOREIGN KEY (raca_id_raca) REFERENCES
raca(id_raca)
);

-- 9. USUARIO_PET (Care Circle Familiar N:M)
CREATE TABLE usuario_pet (
    usuario_id_usuario NUMBER(3) NOT NULL,
    pet_id_pet         NUMBER(3) NOT NULL,
    respon_princ       NUMBER NOT NULL,
    CONSTRAINT usuario_pet_pk PRIMARY KEY (usuario_id_usuario,
pet_id_pet),
    CONSTRAINT ck_usuario_pet_princ CHECK (respon_princ IN (0,
1)),
    CONSTRAINT usuario_pet_usuario_fk FOREIGN KEY
(usuario_id_usuario) REFERENCES usuario(id_usuario),
    CONSTRAINT usuario_pet_pet_fk FOREIGN KEY (pet_id_pet)
REFERENCES pet(id_pet)
);

-- 10. STATUS (Domínio de Tarefas)
```

```
CREATE TABLE status (
    id_status   NUMBER(3) GENERATED BY DEFAULT ON NULL AS
IDENTITY NOT NULL,
    nome_status VARCHAR2(15) NOT NULL,
    CONSTRAINT status_pk PRIMARY KEY (id_status),
    CONSTRAINT ck_status_nome CHECK (nome_status IN ('CONCLUIDO',
'EXPIRADO', 'PENDENTE'))
);

-- 11. TAREFA (Tabela de Fatos de Rotina Gamificada)
CREATE TABLE tarefa (
    id_tarefa          NUMBER(3) GENERATED BY DEFAULT ON NULL AS
IDENTITY NOT NULL,
    titulo             VARCHAR2(30) NOT NULL,
    pontos_tarefa      NUMBER(3) NOT NULL,
    descricao          VARCHAR2(200) NOT NULL,
    criacao            TIMESTAMP NOT NULL,
    prazo              TIMESTAMP NOT NULL,
    conclusao          TIMESTAMP,
    pet_id_pet         NUMBER(3) NOT NULL,
    status_id_status   NUMBER(3) NOT NULL,
    usuario_id_usuario NUMBER(3) NOT NULL,
    CONSTRAINT tarefa_pk PRIMARY KEY (id_tarefa),
    CONSTRAINT ck_tarefa_pontos CHECK (pontos_tarefa BETWEEN 0
AND 10),
    CONSTRAINT tarefa_pet_fk FOREIGN KEY (pet_id_pet) REFERENCES
pet(id_pet),
    CONSTRAINT tarefa_status_fk FOREIGN KEY (status_id_status)
REFERENCES status(id_status),
    CONSTRAINT tarefa_usuario_fk FOREIGN KEY (usuario_id_usuario)
REFERENCES usuario(id_usuario)
);

-- 12. TRILHA (Educação e Adestramento)
CREATE TABLE trilha (
    id_trilha  NUMBER(5) GENERATED BY DEFAULT ON NULL AS IDENTITY
NOT NULL,
    nome       VARCHAR2(30) NOT NULL,
    descricao  VARCHAR2(200) NOT NULL,
```

```
    pet_id_pet NUMBER(3) NOT NULL,
    CONSTRAINT trilha_pk PRIMARY KEY (id_trilha),
    CONSTRAINT trilha_pet_fk FOREIGN KEY (pet_id_pet) REFERENCES
pet(id_pet)
);

-- 13. MODULO
CREATE TABLE modulo (
    id_modulo        NUMBER(5) GENERATED BY DEFAULT ON NULL AS
IDENTITY NOT NULL,
    nome             VARCHAR2(50) NOT NULL,
    tempo_conclusao  VARCHAR2(10) NOT NULL,
    descricao        VARCHAR2(100) NOT NULL,
    trilha_id_trilha NUMBER(5) NOT NULL,
    CONSTRAINT modulo_pk PRIMARY KEY (id_modulo),
    CONSTRAINT modulo_trilha_fk FOREIGN KEY (trilha_id_trilha)
REFERENCES trilha(id_trilha)
);

-- 14. AULA (Conteúdo Pedagógico desacoplado para MongoDB NoSQL)
CREATE TABLE aula (
    id_aula          NUMBER(5) GENERATED BY DEFAULT ON NULL AS
IDENTITY NOT NULL,
    nome             VARCHAR2(50) NOT NULL,
    descricao        VARCHAR2(100) NOT NULL,
    pontos_aula      NUMBER(5) NOT NULL,
    dificuldade      VARCHAR2(20) NOT NULL,
    concluida        NUMBER NOT NULL,
    modulo_id_modulo NUMBER(5) NOT NULL,
    CONSTRAINT aula_pk PRIMARY KEY (id_aula),
    CONSTRAINT ck_aula_concluida CHECK (concluida IN (0, 1)),
    CONSTRAINT aula_modulo_fk FOREIGN KEY (modulo_id_modulo)
REFERENCES modulo(id_modulo)
);

-- 15. HISTORICO (Eventos Clínicos e de Bem-Estar do Pet)
CREATE TABLE historico (
    id_hist    NUMBER(3) GENERATED BY DEFAULT ON NULL AS IDENTITY
NOT NULL,
```

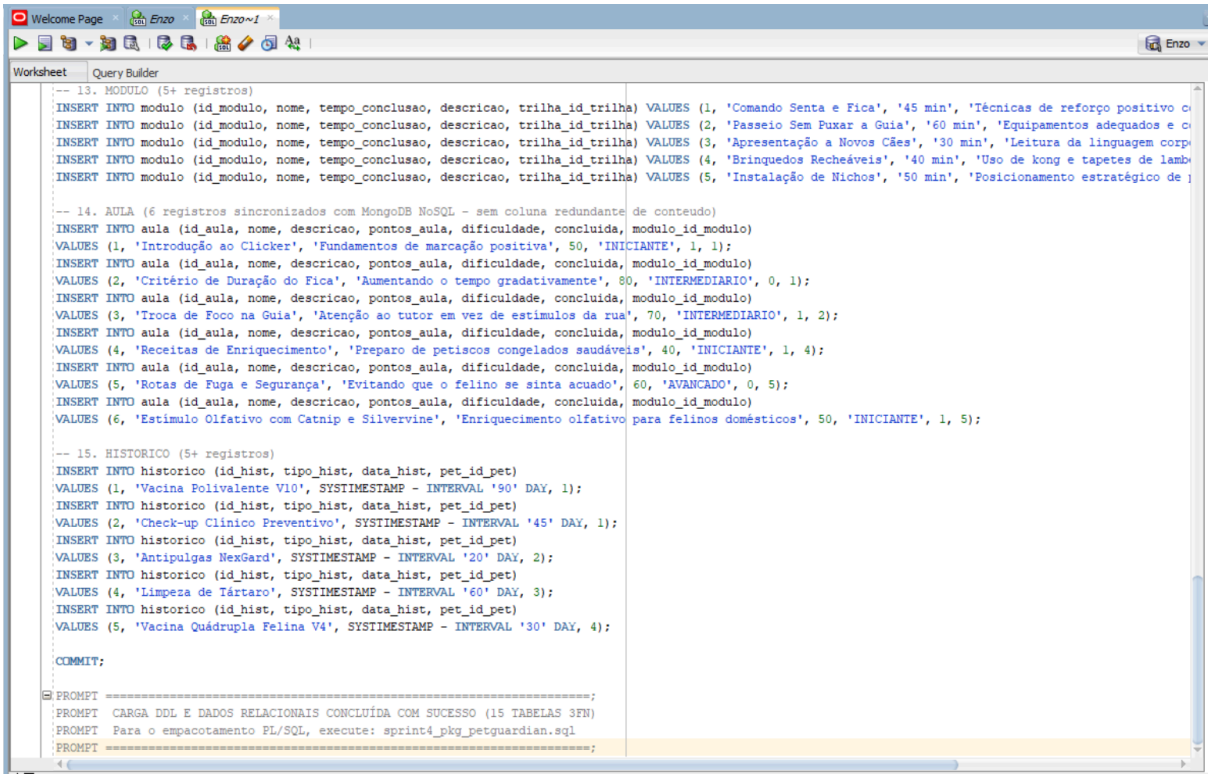
```
    tipo_hist  VARCHAR2(30) NOT NULL,
    data_hist  TIMESTAMP NOT NULL,
    pet_id_pet NUMBER(3) NOT NULL,
    CONSTRAINT historico_pk PRIMARY KEY (id_hist),
    CONSTRAINT historico_pet_fk FOREIGN KEY (pet_id_pet)
REFERENCES pet(id_pet)
);

-- TABELA DE AUDITORIA DML
CREATE TABLE auditoria_dml_tarefa (
    id_auditoria       NUMBER GENERATED ALWAYS AS IDENTITY
PRIMARY KEY,
    nome_usuario       VARCHAR2(50) NOT NULL,
    tipo_operacao      VARCHAR2(10) NOT NULL,
    data_hora_operacao TIMESTAMP DEFAULT SYSTIMESTAMP NOT NULL,
    valores_anteriores VARCHAR2(4000),
    valores_novos      VARCHAR2(4000)
);

-- ÍNDICES EM CHAVES ESTRANGEIRAS (FKs) PARA PERFORMANCE E
PREVENÇÃO DE TABLE LOCKS
CREATE INDEX idx_cidade_estado ON cidade(estado_id_estado);
CREATE INDEX idx_bairro_cidade ON bairro(cidade_id_cidade);
CREATE INDEX idx_endereco_bairro ON endereco(bairro_id_bairro);
CREATE INDEX idx_usuario_endereco ON
usuario(endereco_id_endereco);
CREATE INDEX idx_pet_raca ON pet(raca_id_raca);
CREATE INDEX idx_usuario_pet_pet ON usuario_pet(pet_id_pet);
CREATE INDEX idx_tarefa_pet ON tarefa(pet_id_pet);
CREATE INDEX idx_tarefa_status ON tarefa(status_id_status);
CREATE INDEX idx_tarefa_usuario ON tarefa(usuario_id_usuario);
CREATE INDEX idx_tarefa_status_prazo ON tarefa(status_id_status,
prazo);
CREATE INDEX idx_trilha_pet ON trilha(pet_id_pet);
CREATE INDEX idx_modulo_trilha ON modulo(trilha_id_trilha);
CREATE INDEX idx_aula_modulo ON aula(modulo_id_modulo);
CREATE INDEX idx_historico_pet ON historico(pet_id_pet);
```

```
CREATE UNIQUE INDEX uq_usuario_pet_principal ON usuario_pet (CASE
WHEN respon_princ = 1 THEN pet_id_pet END);
```

DML - Cargas de Dados



```
-- 13. MODULO (5+ registros)
INSERT INTO modulo (id_modulo, nome, tempo_conclusao, descricao, trilha_id_trilha) VALUES (1, 'Comando Senta e Fica', '45 min', 'Técnicas de reforço positivo c
INSERT INTO modulo (id_modulo, nome, tempo_conclusao, descricao, trilha_id_trilha) VALUES (2, 'Passeio Sem Puxar a Guia', '60 min', 'Equipamentos adequados e c
INSERT INTO modulo (id_modulo, nome, tempo_conclusao, descricao, trilha_id_trilha) VALUES (3, 'Apresentação a Novos Cães', '30 min', 'Leitura da linguagem corp
INSERT INTO modulo (id_modulo, nome, tempo_conclusao, descricao, trilha_id_trilha) VALUES (4, 'Brinquedos Recheáveis', '40 min', 'Uso de kong e tapetes de lamb
INSERT INTO modulo (id_modulo, nome, tempo_conclusao, descricao, trilha_id_trilha) VALUES (5, 'Instalação de Nichos', '50 min', 'Posicionamento estratégico de

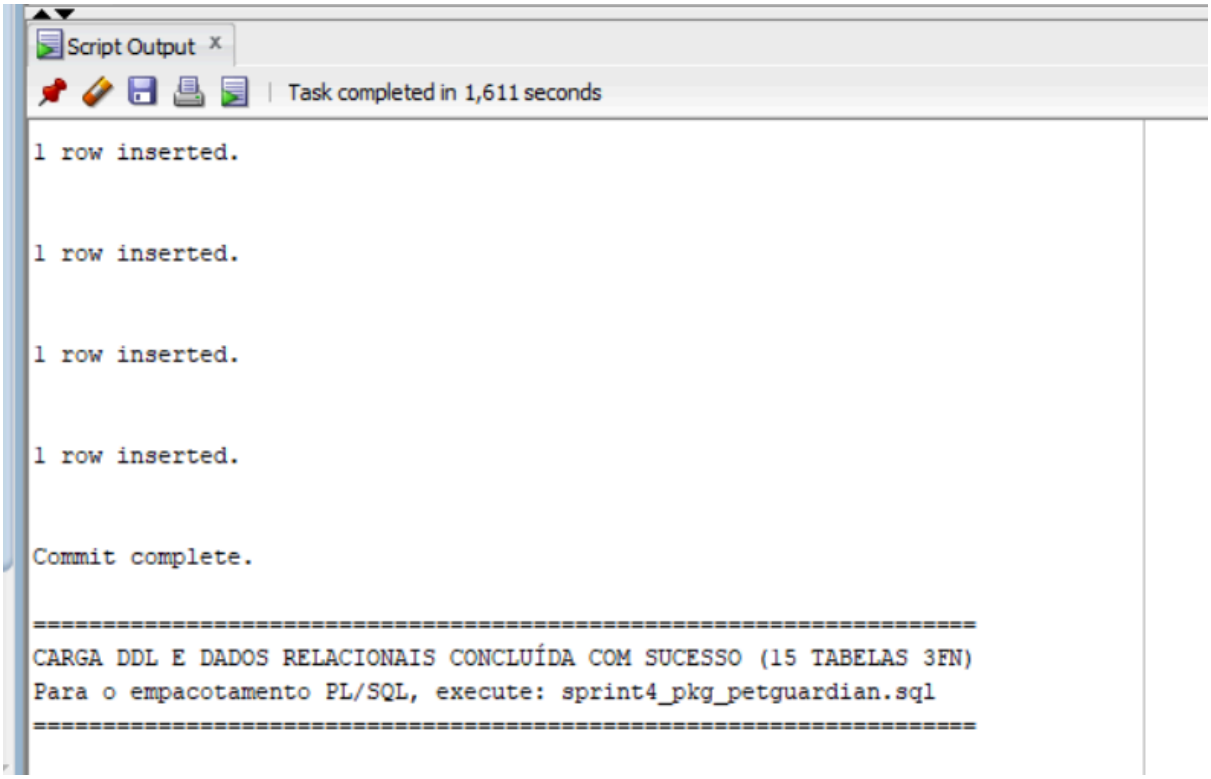
-- 14. AULA (6 registros sincronizados com MongoDB NoSQL - sem coluna redundante de conteudo)
INSERT INTO aula (id_aula, nome, descricao, pontos_aula, dificuldade, concluida, modulo_id_modulo)
VALUES (1, 'Introdução ao Clicker', 'Fundamentos de marcação positiva', 50, 'INICIANTE', 1, 1);
INSERT INTO aula (id_aula, nome, descricao, pontos_aula, dificuldade, concluida, modulo_id_modulo)
VALUES (2, 'Critério de Duração do Fica', 'Aumentando o tempo gradativamente', 80, 'INTERMEDIARIO', 0, 1);
INSERT INTO aula (id_aula, nome, descricao, pontos_aula, dificuldade, concluida, modulo_id_modulo)
VALUES (3, 'Troca de Foco na Guia', 'Atenção ao tutor em vez de estímulos da rua', 70, 'INTERMEDIARIO', 1, 2);
INSERT INTO aula (id_aula, nome, descricao, pontos_aula, dificuldade, concluida, modulo_id_modulo)
VALUES (4, 'Receitas de Enriquecimento', 'Preparo de petiscos congelados saudáveis', 40, 'INICIANTE', 1, 4);
INSERT INTO aula (id_aula, nome, descricao, pontos_aula, dificuldade, concluida, modulo_id_modulo)
VALUES (5, 'Rotas de Fuga e Segurança', 'Evitando que o felino se sinta acuado', 60, 'AVANCADO', 0, 5);
INSERT INTO aula (id_aula, nome, descricao, pontos_aula, dificuldade, concluida, modulo_id_modulo)
VALUES (6, 'Estímulo Olfativo com Catnip e Silvervine', 'Enriquecimento olfativo para felinos domésticos', 50, 'INICIANTE', 1, 5);

-- 15. HISTORICO (5+ registros)
INSERT INTO historico (id_hist, tipo_hist, data_hist, pet_id_pet)
VALUES (1, 'Vacina Polivalente V10', SYSTIMESTAMP - INTERVAL '90' DAY, 1);
INSERT INTO historico (id_hist, tipo_hist, data_hist, pet_id_pet)
VALUES (2, 'Check-up Clínico Preventivo', SYSTIMESTAMP - INTERVAL '45' DAY, 1);
INSERT INTO historico (id_hist, tipo_hist, data_hist, pet_id_pet)
VALUES (3, 'Antipulgas NexGard', SYSTIMESTAMP - INTERVAL '20' DAY, 2);
INSERT INTO historico (id_hist, tipo_hist, data_hist, pet_id_pet)
VALUES (4, 'Limpeza de Tártaro', SYSTIMESTAMP - INTERVAL '60' DAY, 3);
INSERT INTO historico (id_hist, tipo_hist, data_hist, pet_id_pet)
VALUES (5, 'Vacina Quádrupla Felina V4', SYSTIMESTAMP - INTERVAL '30' DAY, 4);

COMMIT;

PROMPT ========================================================================;
PROMPT  CARGA DDL E DADOS RELACIONAIS CONCLUÍDA COM SUCESSO (15 TABELAS 3FN)
PROMPT  Para o empacotamento PL/SQL, execute: sprint4_pkg_petguardian.sql
PROMPT ========================================================================;
```

Output Exibindo Inserções



```
Script Output
Task completed in 1,611 seconds

1 row inserted.


1 row inserted.


1 row inserted.


1 row inserted.


Commit complete.

========================================================================
CARGA DDL E DADOS RELACIONAIS CONCLUÍDA COM SUCESSO (15 TABELAS 3FN)
Para o empacotamento PL/SQL, execute: sprint4_pkg_petguardian.sql
========================================================================
```

Objetos PL/SQL

Empacotamento de Processos: Package PKG_PETGUARDIAN

Na Sprint 4, todas as regras procedurais de negócio do banco relacional foram encapsuladas no pacote corporativo **PKG_PETGUARDIAN**, dividido em **Especificação (PACKAGE)** e **Corpo (PACKAGE BODY)**, garantindo modularidade, segurança transacional e integração direta com o backend:

Subprograma	**Tipo**	**Finalidade de Negócio / Consumo na Aplicação**
pr_calcular_pontuacao_pet	Procedure	Calcula os pontos consolidados de tarefas concluídas e aulas concluídas de um pet específico (GET /pets/{id}/pontos).
pr_calcular_pontuacao_usuario	Procedure	Agrega a pontuação de tarefas e aulas de todos os pets vinculados ao tutor na tabela USUARIO_PET.
pr_obter_resumo_rede_cuidado	Procedure	Consolida em chamada única as métricas do Care Circle do tutor: total de pets, co-cuidadores, tarefas pendentes, concluídas e XP acumulado (GET /usuarios/me/rede-cuidado).
fn_obter_nivel_usuario	Function	Classifica o nível de gamificação do tutor (BRONZE: 0–50 pts, PRATA: 51–150 pts, OURO: 151+ pts) com base no XP acumulado (GET /usuarios/me).
pr_transferir_responsavel_principal	Procedure	Transfere atomicamente a titularidade principal (respon_princ = 1) de um pet para outro membro do Care Circle em USUARIO_PET (PATCH /pets/{id}/responsavel-principal).
trg_audit_tarefa	Trigger DML	Disparada automaticamente (AFTER INSERT OR UPDATE OR DELETE FOR EACH ROW) na tabela TAREFA, gravando usuário, operação, data/hora e estados :OLD e :NEW em AUDITORIA_DML_TAREFA.

Exceções Personalizadas Tratadas no Pacote

Exceção PL/SQL	Código ORA	Condição de Disparo
e_id_invalido	-20018	ID do pet ou do usuário informado é nulo ou menor ou igual a 0.
e_pet_inexistente	-20019	Pet não encontrado na tabela PET.
OTHERS (Score Pet)	-20020	Erro inesperado ao consolidar pontuação do pet.
e_usuario_inexistente	-20022	Usuário não encontrado na tabela USUARIO.
OTHERS (Score Usuário)	-20023	Erro inesperado ao consolidar pontuação do usuário.
OTHERS (Rede Cuidado)	-20024	Erro inesperado ao gerar resumo da rede de cuidado.
e_vinculo_inexistente	-20041	Novo responsável não possui vínculo prévio com o pet em USUARIO_PET.
OTHERS (Transferência)	-20042	Erro inesperado ao transferir responsabilidade principal.

SQL

```sql
SET SERVEROUTPUT ON SIZE UNLIMITED;

--
================================================================
===
-- PARTE 1: ESPECIFICAÇÃO DO PACOTE (PACKAGE SPECIFICATION)
--
================================================================
===
CREATE OR REPLACE PACKAGE pkg_petguardian AS

    -- Exceções públicas com códigos ORA padronizados
    e_id_invalido           EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_id_invalido, -20018);

    e_pet_inexistente       EXCEPTION;
```

```
    PRAGMA EXCEPTION_INIT(e_pet_inexistente, -20019);

    e_usuario_inexistente    EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_usuario_inexistente, -20022);

    e_vinculo_inexistente    EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_vinculo_inexistente, -20041);

    --
-----------------------------------------------------------------
    -- SEÇÃO 1: PONTUAÇÃO GAMIFICADA DE PETS E USUÁRIOS (APP &
JAVA)
    --
-----------------------------------------------------------------

    -- 1.1 Cálculo consolidado de pontuação do pet individual
(tarefas + aulas)
    PROCEDURE pr_calcular_pontuacao_pet (
        p_id_pet         IN NUMBER,
        p_pontos_tarefas OUT NUMBER,
        p_pontos_aulas   OUT NUMBER,
        p_pontos_totais  OUT NUMBER
    );

    -- 1.2 Cálculo agregado de todos os pets do usuário (usado
internamente por pr_obter_resumo_rede_cuidado)
    PROCEDURE pr_calcular_pontuacao_usuario (
        p_id_usuario     IN NUMBER,
        p_pontos_tarefas OUT NUMBER,
        p_pontos_aulas   OUT NUMBER,
        p_pontos_totais  OUT NUMBER
    );

    -- 1.3 Relatório consolidado da rede de cuidado do usuário
(/usuarios/me/rede-cuidado)
    PROCEDURE pr_obter_resumo_rede_cuidado (
        p_id_usuario         IN NUMBER,
        p_total_pets         OUT NUMBER,
        p_total_cuidadores   OUT NUMBER,
```

```
        p_tarefas_pendentes  OUT NUMBER,
        p_tarefas_concluidas OUT NUMBER,
        p_pontos_acumulados  OUT NUMBER
    );

    -- 1.4 Classificação de gamificação do tutor baseada no XP
acumulado (BRONZE, PRATA, OURO)
    FUNCTION fn_obter_nivel_usuario (
        p_id_usuario IN NUMBER
    ) RETURN VARCHAR2;

    --
------------------------------------------------------------------
    -- SEÇÃO 2: OPERAÇÕES TRANSACIONAIS DO APP (REDUÇÃO DE CÓDIGO
JAVA)
    --
------------------------------------------------------------------

    -- 2.1 Transferência atômica de responsabilidade principal no
Care Circle (USUARIO_PET)
    PROCEDURE pr_transferir_responsavel_principal (
        p_id_pet          IN NUMBER,
        p_novo_id_usuario IN NUMBER
    );

END pkg_petguardian;
/

--
================================================================
===
-- PARTE 2: CORPO DO PACOTE (PACKAGE BODY)
--
================================================================
===
CREATE OR REPLACE PACKAGE BODY pkg_petguardian AS

    --
------------------------------------------------------------------
```

```
    -- SEÇÃO 1: PONTUAÇÃO GAMIFICADA DE PETS E USUÁRIOS
    --
-------------------------------------------------------------------

    -- 1.1 Cálculo consolidado de pontuação do pet individual
(tarefas + aulas)
    PROCEDURE pr_calcular_pontuacao_pet (
        p_id_pet         IN NUMBER,
        p_pontos_tarefas OUT NUMBER,
        p_pontos_aulas   OUT NUMBER,
        p_pontos_totais  OUT NUMBER
    ) IS
        v_existe NUMBER;
    BEGIN
        IF p_id_pet IS NULL OR p_id_pet <= 0 THEN
            RAISE_APPLICATION_ERROR(-20018, 'ID do pet fornecido
é nulo ou inválido.');
        END IF;

        SELECT COUNT(1) INTO v_existe FROM pet WHERE id_pet =
p_id_pet;
        IF v_existe = 0 THEN
            RAISE_APPLICATION_ERROR(-20019, 'Pet com id ' ||
p_id_pet || ' não foi encontrado.');
        END IF;

        -- Soma de tarefas concluídas do pet (status_id_status =
2)
        SELECT NVL(SUM(t.pontos_tarefa), 0)
          INTO p_pontos_tarefas
          FROM tarefa t
         WHERE t.pet_id_pet = p_id_pet
           AND t.status_id_status = 2;

        -- Soma de aulas concluídas associadas às trilhas e
módulos do pet
        SELECT NVL(SUM(a.pontos_aula), 0)
          INTO p_pontos_aulas
          FROM aula a
```

```
          JOIN modulo m ON a.modulo_id_modulo = m.id_modulo
          JOIN trilha tr ON m.trilha_id_trilha = tr.id_trilha
         WHERE tr.pet_id_pet = p_id_pet
           AND a.concluida = 1;

        p_pontos_totais := p_pontos_tarefas + p_pontos_aulas;
    EXCEPTION
        WHEN e_id_invalido OR e_pet_inexistente THEN
            RAISE;
        WHEN OTHERS THEN
            RAISE_APPLICATION_ERROR(-20020, 'Erro ao calcular
pontuação consolidada do pet: ' || SQLERRM);
    END pr_calcular_pontuacao_pet;

    -- 1.2 Cálculo agregado de todos os pets do usuário (usado
internamente por pr_obter_resumo_rede_cuidado)
    PROCEDURE pr_calcular_pontuacao_usuario (
        p_id_usuario     IN NUMBER,
        p_pontos_tarefas OUT NUMBER,
        p_pontos_aulas   OUT NUMBER,
        p_pontos_totais  OUT NUMBER
    ) IS
        v_existe NUMBER;
    BEGIN
        IF p_id_usuario IS NULL OR p_id_usuario <= 0 THEN
            RAISE_APPLICATION_ERROR(-20018, 'ID do usuário
fornecido é nulo ou inválido.');
        END IF;

        SELECT COUNT(1) INTO v_existe FROM usuario WHERE
id_usuario = p_id_usuario;
        IF v_existe = 0 THEN
            RAISE_APPLICATION_ERROR(-20022, 'Usuário com id ' ||
p_id_usuario || ' não foi encontrado.');
        END IF;

        -- Tarefas concluídas de todos os pets sob tutela do
usuário (via USUARIO_PET)
        SELECT NVL(SUM(t.pontos_tarefa), 0)
```

```
          INTO p_pontos_tarefas
          FROM tarefa t
         WHERE t.status_id_status = 2
           AND t.pet_id_pet IN (
               SELECT up.pet_id_pet
                 FROM usuario_pet up
                WHERE up.usuario_id_usuario = p_id_usuario
           );

        -- Aulas concluídas de todos os pets sob tutela do
usuário (via USUARIO_PET)
        SELECT NVL(SUM(a.pontos_aula), 0)
          INTO p_pontos_aulas
          FROM aula a
          JOIN modulo m ON a.modulo_id_modulo = m.id_modulo
          JOIN trilha tr ON m.trilha_id_trilha = tr.id_trilha
         WHERE a.concluida = 1
           AND tr.pet_id_pet IN (
               SELECT up.pet_id_pet
                 FROM usuario_pet up
                WHERE up.usuario_id_usuario = p_id_usuario
           );

        p_pontos_totais := p_pontos_tarefas + p_pontos_aulas;
    EXCEPTION
        WHEN e_id_invalido OR e_usuario_inexistente THEN
            RAISE;
        WHEN OTHERS THEN
            RAISE_APPLICATION_ERROR(-20023, 'Erro ao calcular
pontuação agregada do usuário: ' || SQLERRM);
    END pr_calcular_pontuacao_usuario;

    -- 1.3 Relatório consolidado da rede de cuidado do usuário
(/usuarios/me/rede-cuidado)
    PROCEDURE pr_obter_resumo_rede_cuidado (
        p_id_usuario         IN NUMBER,
        p_total_pets         OUT NUMBER,
        p_total_cuidadores   OUT NUMBER,
        p_tarefas_pendentes  OUT NUMBER,
```

```
        p_tarefas_concluidas OUT NUMBER,
        p_pontos_acumulados  OUT NUMBER
    ) IS
        v_existe        NUMBER;
        v_total_tarefas NUMBER;
        v_total_aulas   NUMBER;
    BEGIN
        IF p_id_usuario IS NULL OR p_id_usuario <= 0 THEN
            RAISE_APPLICATION_ERROR(-20018, 'ID do usuário
fornecido é nulo ou inválido.');
        END IF;

        SELECT COUNT(1) INTO v_existe FROM usuario WHERE
id_usuario = p_id_usuario;
        IF v_existe = 0 THEN
            RAISE_APPLICATION_ERROR(-20022, 'Usuário com id ' ||
p_id_usuario || ' não foi encontrado.');
        END IF;

        -- Total de pets tutelados ou co-tutelados pelo usuário
        SELECT NVL(COUNT(DISTINCT up.pet_id_pet), 0)
          INTO p_total_pets
          FROM usuario_pet up
         WHERE up.usuario_id_usuario = p_id_usuario;

        -- Total de co-cuidadores distintos na rede do tutor
        SELECT NVL(COUNT(DISTINCT up2.usuario_id_usuario), 0)
          INTO p_total_cuidadores
          FROM usuario_pet up2
         WHERE up2.usuario_id_usuario <> p_id_usuario
           AND up2.pet_id_pet IN (
               SELECT up.pet_id_pet
                 FROM usuario_pet up
                WHERE up.usuario_id_usuario = p_id_usuario
           );

        -- Tarefas pendentes e concluídas da rede do tutor
(agregação unificada condicional)
```

```
        SELECT NVL(COUNT(CASE WHEN t.status_id_status = 1 THEN 1
END), 0),
               NVL(COUNT(CASE WHEN t.status_id_status = 2 THEN 1
END), 0)
          INTO p_tarefas_pendentes,
               p_tarefas_concluidas
          FROM tarefa t
         WHERE t.pet_id_pet IN (
             SELECT up.pet_id_pet
               FROM usuario_pet up
              WHERE up.usuario_id_usuario = p_id_usuario
         );

        -- Pontuação acumulada total somada (tarefas concluídas +
aulas finalizadas)
        pr_calcular_pontuacao_usuario(p_id_usuario,
v_total_tarefas, v_total_aulas, p_pontos_acumulados);
    EXCEPTION
        WHEN e_id_invalido OR e_usuario_inexistente THEN
            RAISE;
        WHEN OTHERS THEN
            RAISE_APPLICATION_ERROR(-20024, 'Erro ao obter resumo
da rede de cuidado: ' || SQLERRM);
    END pr_obter_resumo_rede_cuidado;

    -- 1.4 Classificação de gamificação do tutor baseada no XP
acumulado (BRONZE, PRATA, OURO)
    FUNCTION fn_obter_nivel_usuario (
        p_id_usuario IN NUMBER
    ) RETURN VARCHAR2 IS
        v_tar NUMBER;
        v_aul NUMBER;
        v_tot NUMBER;
    BEGIN
        IF p_id_usuario IS NULL OR p_id_usuario <= 0 THEN
            RAISE_APPLICATION_ERROR(-20018, 'ID do usuário
fornecido é nulo ou inválido.');
        END IF;
```

```
        pr_calcular_pontuacao_usuario(p_id_usuario, v_tar, v_aul,
v_tot);

        IF v_tot <= 50 THEN
            RETURN 'BRONZE';
        ELSIF v_tot <= 150 THEN
            RETURN 'PRATA';
        ELSE
            RETURN 'OURO';
        END IF;
    END fn_obter_nivel_usuario;

    --
-----------------------------------------------------------------
    -- SEÇÃO 2: OPERAÇÕES TRANSACIONAIS DO APP
    --
-----------------------------------------------------------------

    -- 2.1 Transferência atômica de responsabilidade principal no
Care Circle (USUARIO_PET)
    PROCEDURE pr_transferir_responsavel_principal (
        p_id_pet          IN NUMBER,
        p_novo_id_usuario IN NUMBER
    ) IS
        v_existe_vinculo NUMBER;
    BEGIN
        IF p_id_pet IS NULL OR p_id_pet <= 0 OR p_novo_id_usuario
IS NULL OR p_novo_id_usuario <= 0 THEN
            RAISE_APPLICATION_ERROR(-20018, 'ID do pet e do novo
responsável não podem ser nulos ou inválidos.');
        END IF;

        -- Valida se o novo responsável já faz parte do Care
Circle do pet
        SELECT COUNT(1) INTO v_existe_vinculo
          FROM usuario_pet up
         WHERE up.pet_id_pet = p_id_pet
           AND up.usuario_id_usuario = p_novo_id_usuario;
```

```
        IF v_existe_vinculo = 0 THEN
            RAISE_APPLICATION_ERROR(-20041, 'O novo responsável
(id ' || p_novo_id_usuario || ') não está vinculado ao pet ' ||
p_id_pet || ' em USUARIO_PET.');
        END IF;

        -- Transação atômica otimizada em uma única operação DML
(evita duplo lock e estado intermediário sem titular)
        UPDATE usuario_pet
           SET respon_princ = CASE
                                  WHEN usuario_id_usuario =
p_novo_id_usuario THEN 1
                                  ELSE 0
                              END
         WHERE pet_id_pet = p_id_pet;
    EXCEPTION
        WHEN e_id_invalido OR e_vinculo_inexistente THEN
            RAISE;
        WHEN OTHERS THEN
            RAISE_APPLICATION_ERROR(-20042, 'Erro ao transferir
responsabilidade principal: ' || SQLERRM);
    END pr_transferir_responsavel_principal;

END pkg_petguardian;
/

--
================================================================
===
-- PARTE 3: TRIGGER DE AUDITORIA DML (AUDITORIA_DML_TAREFA)
--
================================================================
===
CREATE OR REPLACE TRIGGER trg_audit_tarefa
AFTER INSERT OR UPDATE OR DELETE ON tarefa
FOR EACH ROW
DECLARE
    v_tipo_operacao VARCHAR2(10);
    v_valores_old   VARCHAR2(4000) := NULL;
```

```
    v_valores_new   VARCHAR2(4000) := NULL;
BEGIN
    IF INSERTING OR UPDATING THEN
        v_valores_new := 'ID_TAREFA=' || :NEW.id_tarefa ||
                         '; TITULO=' || :NEW.titulo ||
                         '; PONTOS=' || :NEW.pontos_tarefa ||
                         '; STATUS=' || :NEW.status_id_status ||
                         '; PET=' || :NEW.pet_id_pet ||
                         '; USUARIO=' || :NEW.usuario_id_usuario;
    END IF;

    IF UPDATING OR DELETING THEN
        v_valores_old := 'ID_TAREFA=' || :OLD.id_tarefa ||
                         '; TITULO=' || :OLD.titulo ||
                         '; PONTOS=' || :OLD.pontos_tarefa ||
                         '; STATUS=' || :OLD.status_id_status ||
                         '; PET=' || :OLD.pet_id_pet ||
                         '; USUARIO=' || :OLD.usuario_id_usuario;
    END IF;

    IF INSERTING THEN
        v_tipo_operacao := 'INSERT';
    ELSIF UPDATING THEN
        v_tipo_operacao := 'UPDATE';
    ELSE
        v_tipo_operacao := 'DELETE';
    END IF;

    INSERT INTO auditoria_dml_tarefa (nome_usuario,
tipo_operacao, data_hora_operacao, valores_anteriores,
valores_novos)
    VALUES (USER, v_tipo_operacao, SYSTIMESTAMP, v_valores_old,
v_valores_new);
END trg_audit_tarefa;
/

--
================================================================
===
```

```
-- PARTE 4: BATERIA DE TESTES DE INTEGRAÇÃO (PKG_PETGUARDIAN)
--
================================================================
===

-- 1. Teste de Pontuação de Pet Individual (Thor - ID 1)
DECLARE
    v_tar NUMBER;
    v_aul NUMBER;
    v_tot NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('=== TESTE 1: pr_calcular_pontuacao_pet
===');
    pkg_petguardian.pr_calcular_pontuacao_pet(1, v_tar, v_aul,
v_tot);
    DBMS_OUTPUT.PUT_LINE('Thor (ID 1) -> Tarefas: ' || v_tar || '
| Aulas: ' || v_aul || ' | Total: ' || v_tot);
END;
/

-- 2. Teste de Pontuação Agregada e Nível de Gamificação do
Usuário (Carlos Silva - ID 1)
DECLARE
    v_tar_usr NUMBER;
    v_aul_usr NUMBER;
    v_tot_usr NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('=== TESTE 2:
pr_calcular_pontuacao_usuario e fn_obter_nivel_usuario ===');
    pkg_petguardian.pr_calcular_pontuacao_usuario(1, v_tar_usr,
v_aul_usr, v_tot_usr);
    DBMS_OUTPUT.PUT_LINE('Carlos Silva (ID 1) -> Tarefas: ' ||
v_tar_usr || ' | Aulas: ' || v_aul_usr || ' | Total: ' ||
v_tot_usr);
    DBMS_OUTPUT.PUT_LINE('Nível de Gamificação (FUNCTION): ' ||
pkg_petguardian.fn_obter_nivel_usuario(1));
END;
/
```

```sql
-- 3. Teste de Resumo da Rede de Cuidado (Carlos Silva - ID 1)
DECLARE
    v_tot_pets NUMBER;
    v_tot_cuid NUMBER;
    v_tar_pend NUMBER;
    v_tar_conc NUMBER;
    v_pts_acum NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('=== TESTE 3:
pr_obter_resumo_rede_cuidado (Carlos Silva ID 1) ===');
    pkg_petguardian.pr_obter_resumo_rede_cuidado(1, v_tot_pets,
v_tot_cuid, v_tar_pend, v_tar_conc, v_pts_acum);
    DBMS_OUTPUT.PUT_LINE('Carlos Silva (ID 1) -> Pets: ' ||
v_tot_pets ||
                         ' | Cuidadores: ' || v_tot_cuid ||
                         ' | Tarefas Pendentes: ' || v_tar_pend
||
                         ' | Tarefas Concluídas: ' || v_tar_conc
||
                         ' | Pontos Acumulados: ' || v_pts_acum);
END;
/

-- 4. Teste de Transferência de Titularidade no Care Circle
BEGIN
    DBMS_OUTPUT.PUT_LINE('=== TESTE 4:
pr_transferir_responsavel_principal ===');
    pkg_petguardian.pr_transferir_responsavel_principal(1, 5);
    DBMS_OUTPUT.PUT_LINE('Titularidade de Thor transferida com
sucesso para Lucas Mendes (ID 5).');
END;
/

-- 5. Disparo e Evidências da Trigger de Auditoria DML
(TRG_AUDIT_TAREFA)
BEGIN
    DBMS_OUTPUT.PUT_LINE('=== TESTE 5: Disparo da Trigger
trg_audit_tarefa na Tabela TAREFA ===');
    UPDATE tarefa
```

```
       SET status_id_status = 2,
           conclusao = SYSTIMESTAMP
     WHERE id_tarefa = 2;
    DBMS_OUTPUT.PUT_LINE('Tarefa ID 2 atualizada com sucesso.
Registro de auditoria gerado pela trigger:');
END;
/

SELECT id_auditoria, nome_usuario, tipo_operacao,
data_hora_operacao, valores_anteriores, valores_novos
  FROM auditoria_dml_tarefa
 ORDER BY id_auditoria ASC;

-- Rollback das operações demonstrativas de teste para preservar
o estado limpo das sementes
ROLLBACK;
```

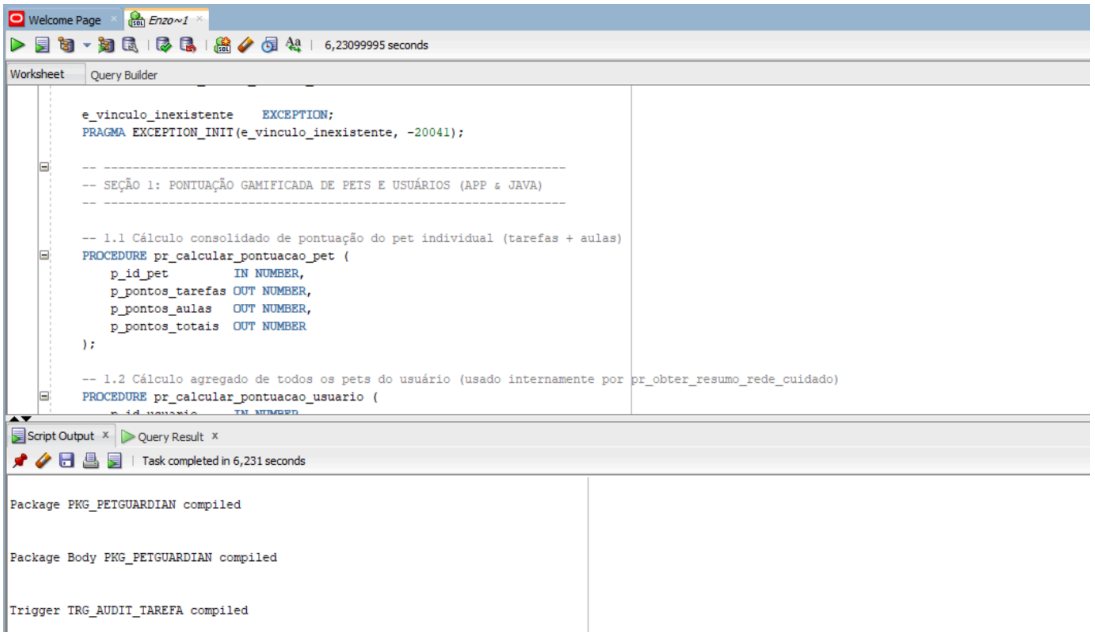
Bateria de Testes e Evidências de Execução (PKG_PETGUARDIAN)

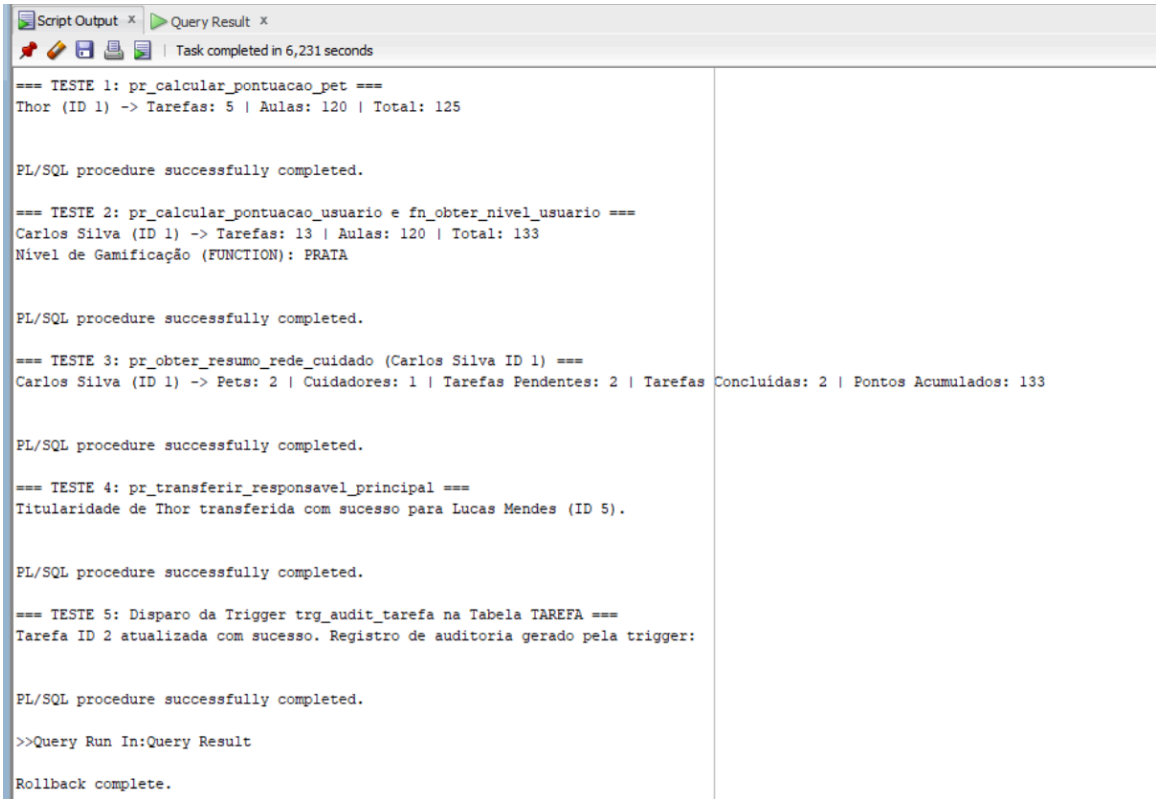
Resultados da Bateria de Testes Integrada (Testes 1 a 5)

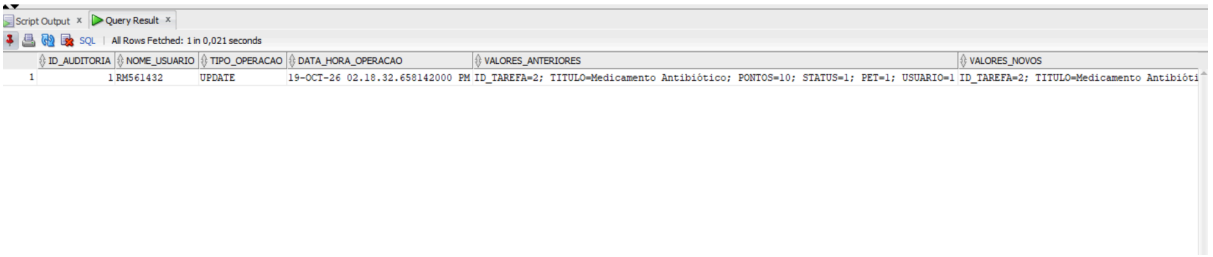
Teste	Subprograma Avaliado	Entrada	Resultado Obtido (DBMS_OUTPUT)
Teste 1	pr_calcular_pontuacao_pet	p_id_pet = 1 (Thor)	Tarefas: 5 — Aulas: 120 — Total: 125
Teste 2	pr_calcular_pontuacao_usuario + fn_obter_nivel_usuario	p_id_usuario = 1 (Carlos Silva)	Tarefas: 13 — Aulas: 120 — Total: 133 ➜ Nível: **PRATA**
Teste 3	pr_obter_resumo_rede_cuidado	p_id_usuario = 1 (Carlos Silva)	Pets: 2 — Cuidadores: 1 — Pendentes: 2 — Concluídas: 2 — Pontos: 133
Teste 4	pr_transferir_responsavel_principal	p_id_pet = 1, p_novo_id_usuario = 5	Titularidade de Thor transferida para Lucas Mendes (ID 5).

Teste	Subprograma Avaliado	Entrada	Resultado Obtido (DBMS_OUTPUT)
Teste 5	trg_audit_tarefa	UPDATE tarefa SET status_id_status = 2 WHERE id_tarefa = 2	Registro de auditoria UPDATE gravado automaticamente em AUDITORIA_DML_TAREFA com :OLD e :NEW.

Prints:







Banco de Dados Não Relacional — MongoDB (conteudos_aula)

Persistência Poliglota e Modelagem Orientada a Documentos

Seguindo os princípios de persistência poliglota, os metadados estruturados de progresso e pontuação permanecem na tabela relacional AULA no Oracle (3FN), enquanto o conteúdo pedagógico extenso em Markdown e listas de materiais multimídia (links_recursos) foram desacoplados para a coleção **conteudos_aula** no **MongoDB Atlas**, referenciando a aula relacional por meio da chave aula_id.

1. Dataset Estruturado em JSON (conteudos_dataset.json)

JSON

```
[
  {
    "aula_id": 1,
    "tipo_conteudo": "PASSO_A_PASSO",
    "corpo_markdown": "# Introdução ao Clicker\n\nFundamentos de
marcação positiva utilizando o método de indução positiva com
petisco.\n\n### Passo a Passo:\n1. Carregue o clicker com petisco
de alto valor.\n2. Clique no exato instante em que o cão oferecer
a resposta desejada.\n3. Entregue o petisco em até 2 segundos
para fixação cognitiva.\n\n> **Dica de Ouro:** Repita em sessões
de no máximo 3 minutos para não causar estresse cognitivo.",
    "links_recursos": [

"https://www.akc.org/expert-advice/training/clicker-training-your
-dog-mark-and-reward/",
      "https://en.wikipedia.org/wiki/Clicker_training"
    ],
    "atualizado_em": "2026-10-08T18:00:00Z"
  },
  {
```

```
    "aula_id": 2,
    "tipo_conteudo": "ARTIGO",
    "corpo_markdown": "# Critério de Duração do
Fica\n\nAumentando o tempo gradativamente sem que o cão quebre a
posição de espera.\n\n### Princípios Fundamentais:\n- Inicie com
3 segundos de permanência.\n- Dê um passo para trás e retorne
antes de liberar o animal com comando de liberação ('Ok!').\n-
Jamais chame o cão até você quando ele estiver no comando Fica
para não gerar antecipação de quebra de postura.",
    "links_recursos": [

"https://www.rspca.org.uk/adviceandwelfare/pets/dogs/training/sta
y"
    ],
    "atualizado_em": "2026-10-08T18:00:00Z"
  },
  {
    "aula_id": 3,
    "tipo_conteudo": "VIDEO",
    "corpo_markdown": "# Troca de Foco na Guia\n\nAtenção ao
tutor em vez de estímulos da rua durante os passeios
urbanos.\n\n### Procedimento Prático:\n1. Use guia de 1,5m sem
tensionar.\n2. Sempre que avistar estímulo concorrente na
calçada, chame o cão e recompense o contato visual imediato.\n3.
Mantenha caminhada fluida e recompense o caminhar ao lado.",
    "links_recursos": [

"https://www.akc.org/expert-advice/training/teach-puppy-walk-leas
h/"
    ],
    "atualizado_em": "2026-10-08T18:00:00Z"
  },
  {
    "aula_id": 4,
    "tipo_conteudo": "PASSO_A_PASSO",
    "corpo_markdown": "# Receitas de Enriquecimento\n\nPreparo de
petiscos congelados saudáveis para enriquecimento ambiental e
redução de ansiedade de separação.\n\n### Receita Kong
Calmante:\n- 1 colher de pasta de amendoim 100% integral (sem
```

```
xilitol e sem açúcar).\n- Frutas picadas permitidas (maçã sem
semente e banana amassada).\n- Congele por 4 horas no freezer
antes de oferecer ao animal.",
    "links_recursos": [
      "https://www.kongcompany.com/recipes/"
    ],
    "atualizado_em": "2026-10-08T18:00:00Z"
  },
  {
    "aula_id": 5,
    "tipo_conteudo": "ARTIGO",
    "corpo_markdown": "# Rotas de Fuga e Segurança\n\nEvitando
que o felino se sinta acuado em ambientes verticais para
felinos.\n\n### Recomendações de Instalação:\n- Instale
prateleiras e nichos a pelo menos 1,5m de altura com superfície
antiderrapante.\n- Nunca crie rotas sem saída ('becos sem saída
verticais') em lares com mais de um gato.\n- Garanta redes de
proteção testadas em 100% das janelas e sacadas.",
    "links_recursos": [

"https://www.vet.cornell.edu/departments-centers-and-institutes/c
ornell-feline-health-center/health-information/feline-health-topi
cs/feline-behavior-problems-destructive-behavior"
    ],
    "atualizado_em": "2026-10-08T18:00:00Z"
  },
  {
    "aula_id": 6,
    "tipo_conteudo": "ARTIGO",
    "corpo_markdown": "# Estímulo Olfativo com Catnip e
Silvervine\n\nEnriquecimento olfativo para felinos domésticos
para estímulo mental e prevenção de apatia.\n\n### Guia de
Aplicação:\n- **Catnip (Nepeta cataria):** aplicar 1 a 2 vezes
por semana em brinquedos ou arranhadores.\n- **Silvervine
(Actinidia polygama):** opção consagrada para os 30% de felinos
que não respondem ao Catnip.\n- Guarde os brinquedos em potes
herméticos após a brincadeira para preservar os óleos
essenciais.",
    "links_recursos": [
```

```
      "https://en.wikipedia.org/wiki/Catnip",
      "https://en.wikipedia.org/wiki/Actinidia_polygama"
    ],
    "atualizado_em": "2026-10-08T18:00:00Z"
  }
]
```

2. Script de Criação da Coleção, Validação de Schema, Índices e Carga (setup_mongodb_conteudos.js)

JavaScript

```
// Função de log universal (compatível com mongosh, mongo shell e Node.js)
const log = typeof console !== "undefined" && console.log ? console.log : print;

// 1. Seleciona ou cria o banco de dados oficial da aplicação
const dbName = "petguardian";
const dbTarget = typeof db !== "undefined" && db.getSiblingDB ? db.getSiblingDB(dbName) : db;

log(">>> Conectado ao banco: " + dbName);

// 2. Remove coleção preexistente se necessário para execução limpa e repetível
try {
  dbTarget.conteudos_aula.drop();
  log(">>> Coleção antiga 'conteudos_aula' removida para recriação limpa.");
} catch (e) {
  // Ignora erro se a coleção ainda não existia
}

// 3. Criação da Coleção com Validação de Schema (JSON Schema Validator)
// Garante integridade estrutural e tipagem dos documentos NoSQL
```

```
dbTarget.createCollection("conteudos_aula", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["aula_id", "tipo_conteudo", "corpo_markdown"],
      properties: {
        aula_id: {
          bsonType: ["int", "long", "string"],
          description: "ID relacional da aula de origem no Oracle
(inteiro/long ou GUID string obrigatório e único)."
        },
        tipo_conteudo: {
          bsonType: "string",
          description: "Classificação do formato pedagógico (ex:
ARTIGO, VIDEO, PASSO_A_PASSO)."
        },
        corpo_markdown: {
          bsonType: "string",
          description: "Conteúdo rico formatado em Markdown, sem
restrição de tamanho."
        },
        links_recursos: {
          bsonType: "array",
          description: "Lista de URLs para materiais
complementares e vídeos de apoio.",
          items: {
            bsonType: "string"
          }
        },
        atualizado_em: {
          bsonType: ["string", "date"],
          description: "Data/hora ISO da última revisão do
conteúdo."
        }
      }
    }
  }
});
```

```
log(">>> Coleção 'conteudos_aula' criada com JSON Schema
Validator.");

// 4. Criação de Índices Estratégicos para Otimização NoSQL
// Índice 1: Unicidade no ID da aula relacional (idempotência e
integridade lógica)
dbTarget.conteudos_aula.createIndex(
  { "aula_id": 1 },
  { unique: true, name: "idx_aula_id_unique" }
);

// Índice 2: Índice Textual para buscas por palavras-chave dentro
do conteúdo pedagógico
dbTarget.conteudos_aula.createIndex(
  { "corpo_markdown": "text" },
  { name: "idx_busca_textual_conteudo" }
);

// Índice 3: Índice de filtro por formato de conteúdo
dbTarget.conteudos_aula.createIndex(
  { "tipo_conteudo": 1 },
  { name: "idx_tipo_conteudo" }
);

log(">>> Índices estratégicos criados com sucesso.");

// 5. Carga dos Dados a partir do Dataset Estruturado JSON
(conteudos_dataset.json)
let datasetConteudos = [];

try {
  // Leitura dinâmica do arquivo exportado do banco relacional
  const fs = require("fs");
  const path = require("path");
  const caminhoJson = path.resolve(typeof __dirname !==
"undefined" ? __dirname : ".", "conteudos_dataset.json");

  if (fs.existsSync(caminhoJson)) {
    const conteudoBruto = fs.readFileSync(caminhoJson, "utf8");
```

```
    const dadosParsed = JSON.parse(conteudoBruto);

    datasetConteudos = dadosParsed.map(item => ({
      ...item,
      aula_id: typeof NumberInt !== "undefined" ?
NumberInt(item.aula_id) : item.aula_id,
      atualizado_em: item.atualizado_em || new
Date().toISOString()
    }));
    log(">>> Sucesso: Dataset carregado dinamicamente a partir de
'conteudos_dataset.json' (" + datasetConteudos.length + "
documentos).");
  } else {
    throw new Error("Arquivo 'conteudos_dataset.json' não
encontrado no diretório.");
  }
} catch (erroCarregamento) {
  log(">>> Aviso: Leitura via sistema de arquivos indisponível no
cliente (" + erroCarregamento.message + "). Utilizando dataset
estruturado integrado.");

  datasetConteudos = [
    {
      aula_id: typeof NumberInt !== "undefined" ? NumberInt(1) :
1,
      tipo_conteudo: "PASSO_A_PASSO",
      corpo_markdown: "# Introdução ao Clicker\n\nFundamentos de
marcação positiva utilizando o método de indução positiva com
petisco.\n\n### Passo a Passo:\n1. Carregue o clicker com petisco
de alto valor.\n2. Clique no exato instante em que o cão oferecer
a resposta desejada.\n3. Entregue o petisco em até 2 segundos
para fixação cognitiva.\n\n> **Dica de Ouro:** Repita em sessões
de no máximo 3 minutos para não causar estresse cognitivo.",
      links_recursos: [

"https://www.akc.org/expert-advice/training/clicker-training-your
-dog-mark-and-reward/",
        "https://en.wikipedia.org/wiki/Clicker_training"
      ],
```

```
      atualizado_em: "2026-10-08T18:00:00Z"
    },
    {
      aula_id: typeof NumberInt !== "undefined" ? NumberInt(2) :
2,
      tipo_conteudo: "ARTIGO",
      corpo_markdown: "# Critério de Duração do
Fica\n\nAumentando o tempo gradativamente sem que o cão quebre a
posição de espera.\n\n### Princípios Fundamentais:\n- Inicie com
3 segundos de permanência.\n- Dê um passo para trás e retorne
antes de liberar o animal com comando de liberação ('Ok!').\n-
Jamais chame o cão até você quando ele estiver no comando Fica
para não gerar antecipação de quebra de postura.",
      links_recursos: [

"https://www.rspca.org.uk/adviceandwelfare/pets/dogs/training/sta
y"
      ],
      atualizado_em: "2026-10-08T18:00:00Z"
    },
    {
      aula_id: typeof NumberInt !== "undefined" ? NumberInt(3) :
3,
      tipo_conteudo: "VIDEO",
      corpo_markdown: "# Troca de Foco na Guia\n\nAtenção ao
tutor em vez de estímulos da rua durante os passeios
urbanos.\n\n### Procedimento Prático:\n1. Use guia de 1,5m sem
tensionar.\n2. Sempre que avistar estímulo concorrente na
calçada, chame o cão e recompense o contato visual imediato.\n3.
Mantenha caminhada fluida e recompense o caminhar ao lado.",
      links_recursos: [

"https://www.akc.org/expert-advice/training/teach-puppy-walk-leas
h/"
      ],
      atualizado_em: "2026-10-08T18:00:00Z"
    },
    {
```

```
      aula_id: typeof NumberInt !== "undefined" ? NumberInt(4) :
4,
      tipo_conteudo: "PASSO_A_PASSO",
      corpo_markdown: "# Receitas de Enriquecimento\n\nPreparo de
petiscos congelados saudáveis para enriquecimento ambiental e
redução de ansiedade de separação.\n\n### Receita Kong
Calmante:\n- 1 colher de pasta de amendoim 100% integral (sem
xilitol e sem açúcar).\n- Frutas picadas permitidas (maçã sem
semente e banana amassada).\n- Congele por 4 horas no freezer
antes de oferecer ao animal.",
      links_recursos: [
        "https://www.kongcompany.com/recipes/"
      ],
      atualizado_em: "2026-10-08T18:00:00Z"
    },
    {
      aula_id: typeof NumberInt !== "undefined" ? NumberInt(5) :
5,
      tipo_conteudo: "ARTIGO",
      corpo_markdown: "# Rotas de Fuga e Segurança\n\nEvitando
que o felino se sinta acuado em ambientes verticais para
felinos.\n\n### Recomendações de Instalação:\n- Instale
prateleiras e nichos a pelo menos 1,5m de altura com superfície
antiderrapante.\n- Nunca crie rotas sem saída ('becos sem saída
verticais') em lares com mais de um gato.\n- Garanta redes de
proteção testadas em 100% das janelas e sacadas.",
      links_recursos: [

"https://www.vet.cornell.edu/departments-centers-and-institutes/c
ornell-feline-health-center/health-information/feline-health-topi
cs/feline-behavior-problems-destructive-behavior"
      ],
      atualizado_em: "2026-10-08T18:00:00Z"
    },
    {
      aula_id: typeof NumberInt !== "undefined" ? NumberInt(6) :
6,
      tipo_conteudo: "ARTIGO",
```

```
      corpo_markdown: "# Estímulo Olfativo com Catnip e
Silvervine\n\nEnriquecimento olfativo para felinos domésticos
para estímulo mental e prevenção de apatia.\n\n### Guia de
Aplicação:\n- **Catnip (Nepeta cataria):** aplicar 1 a 2 vezes
por semana em brinquedos ou arranhadores.\n- **Silvervine
(Actinidia polygama):** opção consagrada para os 30% de felinos
que não respondem ao Catnip.\n- Guarde os brinquedos em potes
herméticos após a brincadeira para preservar os óleos
essenciais.",
      links_recursos: [
        "https://en.wikipedia.org/wiki/Catnip",
        "https://en.wikipedia.org/wiki/Actinidia_polygama"
      ],
      atualizado_em: "2026-10-08T18:00:00Z"
    }
  ];
}

const resultadoCarga =
dbTarget.conteudos_aula.insertMany(datasetConteudos);
log(">>> Carga executada! Documentos inseridos com sucesso: " +
(resultadoCarga.insertedCount ||
Object.keys(resultadoCarga.insertedIds || {}).length));

// 6. Consulta de Verificação com Projeção
log(">>> Consulta de teste (Conteúdos cadastrados):");
dbTarget.conteudos_aula.find({}, {
  aula_id: 1,
  tipo_conteudo: 1,
  atualizado_em: 1
}).forEach(doc => {
  log(" - Aula ID: " + doc.aula_id + " | Formato: " +
doc.tipo_conteudo);
});
```

3. Evidência de Execução e Importação no MongoDB (MONGOSH)

